Student Course Management System

Project Report

Submitted by:

Hanem Atef Elsayed

Student ID: 12262

GitHub Repository:

https://github.com/HanemAtef/cpp_project.git

Table of Contents

Table of Contents 2
1. Introduction 4
2. Project Overview 4
3. Project Beneficiaries 4
4. Detailed Project Description 4
4.1 System Architecture 4
4.2 OOP Principles Applied 5
Inheritance 5
Encapsulation 5
Data Structures (STL) 5
5. Detailed Analysis 5
5.1 Problem Statement 5
5.2 Proposed Solution 5
5.3 Limitations 6
6. Task Breakdown Structure 6
6.1 Major Tasks 6
7. Time Plan (Gantt Chart) 6
8. System Architecture and Design 6
8.1 Execution Flowchart 6
8.2 UML Class Diagram 7
9. Key Code Snippets 8
9.1 Student Enrollment with Duplicate Check 8
9.2 GPA Sorting with Custom Comparator 8
9.3 Input Validation in Main Loop 9
10. Testing Scenarios and Results 9
11. End-User Guide 9
11.1 System Requirements 10
11.2 Compilation 10
11.3 Running the Application 10
11.4 Menu Options 10
11.5 Sample Session 10
12. Conclusion 10
13. References 11
14. Appendices 11

Appendix A: File Structure 11
Appendix B: Class Relationship Summary 11

1. Introduction

The Student Course Management System is a console-based C++ application designed to help educational administrators efficiently manage student records and course enrollments. Built using the principles of Object-Oriented Programming (OOP) and the C++ Standard Template Library (STL), this system provides a straightforward menu-driven interface for performing all essential student data operations.

This report documents the complete development lifecycle of the project, covering the problem analysis, system design, implementation details, testing results, and a user guide. The goal of this project is to demonstrate practical application of C++ concepts including inheritance, encapsulation, and STL data structures in a real-world scenario.

2. Project Overview

The system allows an administrator to perform the following core operations through an interactive text-based menu:

- **Add / Remove Students:** Manage the student registry with full validation.
- **Search Students:** Quickly locate a student record by their unique ID.
- **Display All Students:** View a complete listing of all registered students.
- **Course Enrollment:** Assign courses to students, with automatic duplicate prevention.
- **View Student Courses:** Display all courses a specific student is enrolled in.
- **Sort by GPA:** Rank all students from highest to lowest GPA.
- **Error Handling:** Gracefully handles invalid inputs without crashing.

The application runs as a loop, continuously presenting the menu until the user chooses to exit (Option 8). Input validation is enforced at every step to ensure data integrity.

3. Project Beneficiaries

This system is designed to serve the following stakeholders:

- **Academic Administrators:** Staff responsible for managing student enrollment and academic records benefit from a fast, reliable tool to add, remove, and search student data.
- **Faculty Members:** Instructors can use the system to view which students are enrolled in their courses and track academic performance via GPA sorting.
- **Students:** Indirectly, students benefit from an organized system that accurately tracks their enrollment and academic standing.
- **Software Developers (Educational):** This project serves as a practical learning example for OOP, STL usage, and C++ best practices.

4. Detailed Project Description

4.1 System Architecture

The system is structured across five source files, adhering to a clean separation of concerns:

File	Class	Responsibility
Person.h / Person.cpp	Person	Base class with the student name attribute
Student.h / Student.cpp	Student	Derived class with ID, GPA, and course enrollment logic
Manager.h / Manager.cpp	Manager	Manages the student map and all CRUD operations
Main.cpp	—	Entry point; runs the menu loop

4.2 OOP Principles Applied

Inheritance

The Person class serves as the base class, providing the name attribute and the getName() accessor method. The Student class inherits from Person using public inheritance and extends it with academic-specific data: a unique integer ID, a float GPA value, and a set of enrolled course names.

Encapsulation

All class attributes are declared private or protected, and are only accessible through well-defined public getter methods (getId(), getGpa(), getCourses()) and controlled mutation functions (enrollCourse()). This ensures that the internal state of each object cannot be corrupted by external code.

Data Structures (STL)

- **std::map<int, Student>:** Used in Manager for O(log n) insertion, deletion, and lookup by Student ID. Automatically keeps entries sorted by key.
- **std::set<string>:** Used in Student to store course names. Automatically prevents duplicate course enrollments.
- **std::vector + std::sort:** Used in sortStudentsByGPA() to create a temporary sortable copy of the map, enabling custom comparison without modifying the primary data store.

5. Detailed Analysis

5.1 Problem Statement

Managing student records manually or with flat-file approaches leads to slow lookups, data redundancy, and the risk of data corruption. The challenge is to implement an efficient, scalable system that enforces data uniqueness, prevents logical errors (like duplicate enrollments), and remains easy to interact with through a simple interface.

5.2 Proposed Solution

The solution leverages C++ STL containers chosen specifically for their algorithmic properties. A std::map guarantees O(log n) search and insertion performance for student lookups by ID, which is

superior to linear search in an array or vector. A std::set for courses guarantees uniqueness and O(log n) insertion check, eliminating the need for manual duplicate detection.

5.3 Limitations

- Data is stored in-memory only and is not persisted between sessions. Restarting the program clears all data.
- The system operates via a command-line interface only; no graphical user interface is provided.
- The Manager class does not currently support editing a student's GPA or name after creation.
- No file I/O or database backend is implemented for persistent storage.

6. Task Breakdown Structure

6.1 Major Tasks

1. Requirements Analysis: Define system features, constraints, and data structures needed.
2. OOP Design: Design the class hierarchy (Person → Student, Manager) and define all interfaces.
3. UML Diagramming: Create UML class diagram showing relationships between classes.
4. Implementation — Person & Student classes: Code the base and derived class with all attributes and methods.
5. Implementation — Manager class: Implement all seven management functions with full validation.
6. Implementation — Main Loop: Build the menu system with input validation and switch-case dispatch.
7. Testing: Execute test cases covering normal flow, edge cases, and error scenarios.
8. Flowchart Creation: Create the execution flowchart documenting program control flow.
9. Documentation & Report Writing: Compile findings, diagrams, and test results into this report.

7. Time Plan (Gantt Chart)

Task	Week 1	Week 2	Week 3	Week 4	Week 5
Requirements Analysis	✓				
OOP Design & UML		✓			
Implementation		✓	✓		
Testing & Debugging			✓	✓	
Documentation & Report				✓	✓

Figure 1: Project Gantt Chart — Task schedule over 5 weeks

8. System Architecture and Design

8.1 Execution Flowchart

The following flowchart illustrates the complete control flow of the application, from program initialization through the main menu loop to each operation and exit condition.

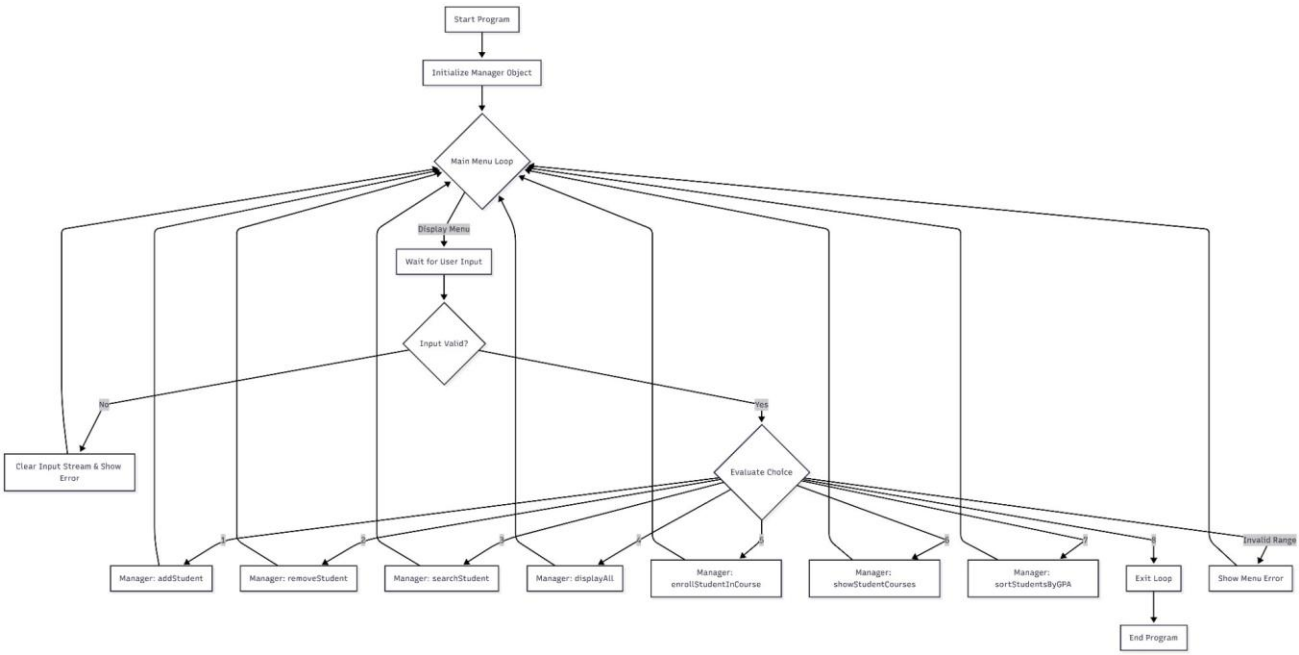

Figure 2: Program Execution Flowchart

8.2 UML Class Diagram

The UML diagram below shows the structural relationships between the three core classes. Person is the base class, Student inherits from Person, and Manager composes Student objects through a map container.

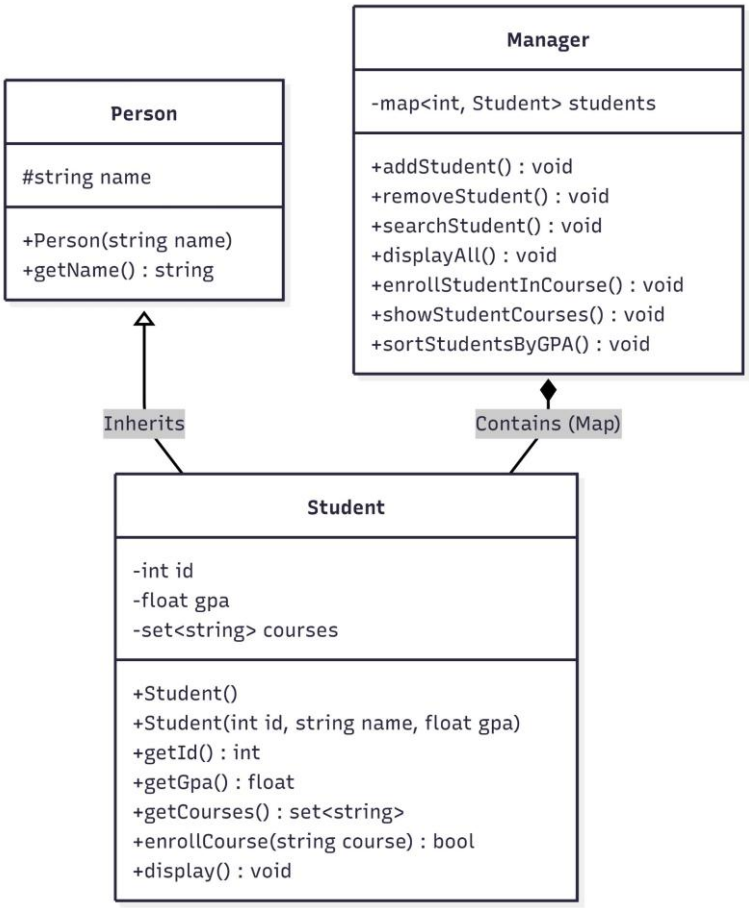

Figure 3: UML Class Diagram

9. Key Code Snippets

9.1 Student Enrollment with Duplicate Check

The enrollCourse() method uses std::set's count() to check for duplicate enrollment in O(log n) time:

```
bool Student::enrollCourse(string course) {
    if (courses.count(course)) {    // O(log n) duplicate check
        return false;               // Already enrolled
    }
    courses.insert(course);         // O(log n) insertion
    return true;
}
```

9.2 GPA Sorting with Custom Comparator

Students are sorted by copying the map to a vector and applying std::sort with a custom comparator:

```
bool compareByGPA(pair<int, Student>& a, pair<int, Student>& b) {
    return a.second.getGpa() > b.second.getGpa();
}

void Manager::sortStudentsByGPA() {
    vector<pair<int, Student>> temp(students.begin(), students.end());
    sort(temp.begin(), temp.end(), compareByGPA);
    for (pair<int, Student> element : temp)
```

```
        element.second.display();
}
```

9.3 Input Validation in Main Loop

The main loop validates cin input to prevent crashes from non-numeric input:

```
if (cin.fail()) {
    cin.clear();
    cin.ignore(1000, '\n');
    cout << "Invalid input! Please enter a number." << endl;
    continue;
}
```

10. Testing Scenarios and Results

A total of 10 test cases were executed to verify correctness of all major functions. The table below summarizes the inputs, expected outputs, and pass/fail results.

#	Test Case	Input	Expected Output	Result
1	Add Valid Student	ID:101, Name: Ali, GPA:3.5	Student added successfully	PASS
2	Add Duplicate ID	ID:101 (again)	Student already exists	PASS
3	Invalid GPA	GPA: 5.0	Invalid GPA!	PASS
4	Remove Existing Student	ID:101	Student removed	PASS
5	Remove Non-existent	ID:999	Student not found	PASS
6	Search Student	ID:102	Display student info	PASS
7	Enroll in Course	ID:102, Math	Enrolled successfully	PASS
8	Duplicate Enrollment	ID:102, Math	Already enrolled	PASS
9	Sort by GPA	Multiple students	Sorted from high to low	PASS
10	Invalid Menu Input	Enter 'abc'	Error message, no crash	PASS

Table 1: Test Cases and Results — All 10 tests passed successfully

All test cases passed successfully. No crashes or undefined behaviors were encountered during testing. The input validation correctly handles both non-numeric inputs and out-of-range menu choices.

11. End-User Guide

11.1 System Requirements

- **Compiler:** Any C++11 or later compliant compiler (e.g., GCC, MSVC, Clang)
- **Operating System:** Windows, Linux, or macOS
- **IDE (Optional):** Visual Studio, VS Code with C++ extension, or any text editor

11.2 Compilation

Compile all source files together using the following command:

```
g++ -o StudentSystem Main.cpp Manager.cpp Student.cpp Person.cpp
```

On Windows with MSVC, use: cl Main.cpp Manager.cpp Student.cpp Person.cpp

11.3 Running the Application

```
./StudentSystem     # Linux/macOS
StudentSystem.exe   # Windows
```

11.4 Menu Options

Upon launch, the system waits for a numeric input (1–8). The available operations are:

10. Add Student — Enter a unique ID, full name, and GPA (0.0–4.0).
11. Remove Student — Enter the student ID to permanently remove from the system.
12. Search Student — Enter an ID to display that student's full details.
13. Display All Students — Lists every student currently in the system.
14. Enroll Student in Course — Enter student ID then course name to enroll.
15. Show Student Courses — Displays all courses a specific student is enrolled in.
16. Sort Students by GPA — Prints all students ranked highest to lowest GPA.
17. Exit — Terminates the program cleanly.

11.5 Sample Session

```
Enter your choice: 1
Enter Student ID: 101
Enter Name: Hager Sherif
Enter GPA: 3.8
Student  Hager Sherif  added successfully!

Enter your choice: 5
Enter Student ID: 101
Enter Course Name: Data Structures
Enrolled to Course Successfully!

Enter your choice: 7
Students From Highest to Lowest GPA:
ID: 101  Name: Hager Sherif  GPA: 3.8
```

12. Conclusion

The Student Course Management System successfully demonstrates the application of core C++ programming concepts including OOP design, inheritance, encapsulation, and efficient data management using the STL. The system fulfills all specified requirements: it allows administrators to manage students and course enrollments through a validated, menu-driven interface.

The deliberate choice of std::map for the student container and std::set for course storage ensures that the system performs at O(log n) for all critical operations, making it scalable beyond small datasets. The input validation mechanism ensures robustness in real-world usage.

Future enhancements could include file-based persistence (reading/writing student data to CSV or binary files), a graphical user interface, support for updating student records, and a more comprehensive role-based access system distinguishing between administrator and read-only views.

13. References

18. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
19. cppreference.com. (2024). std::map — C++ Reference. https://en.cppreference.com/w/cpp/container/map
20. cppreference.com. (2024). std::set — C++ Reference. https://en.cppreference.com/w/cpp/container/set
21. cppreference.com. (2024). std::sort — C++ Reference. https://en.cppreference.com/w/cpp/algorithm/sort
22. Lippman, S., Lajoie, J., & Moo, B. (2012). C++ Primer (5th ed.). Addison-Wesley Professional.
23. ISO/IEC 14882:2011. (2011). Information technology — Programming languages — C++.

14. Appendices

Appendix A: File Structure

```
C--Final-project/
├── Main.cpp        — Entry point and menu loop
├── Person.h        — Base class declaration
├── Person.cpp      — Base class implementation
├── Student.h       — Student class declaration
├── Student.cpp     — Student class implementation
├── Manager.h       — Manager class declaration
├── Manager.cpp     — Manager class implementation
├── FlowChart.png   — Execution flowchart
├── Uml.png         — UML class diagram
└── README.md       — Compilation & usage guide
```

Appendix B: Class Relationship Summary

- **Person → Student:** Public inheritance. Student IS-A Person.
- **Manager ◆→ Student:** Composition via std::map. Manager HAS-MANY Students.
- **Student ◆→ courses:** Composition via std::set. Student HAS-MANY course strings.